

Implementação de Curvas Paramétricas Cúbicas Bézier e Hermite

Arthur Oliveira Santos

23 de junho de 2026

1 Introdução

Este trabalho apresenta a implementação de duas curvas paramétricas cúbicas em um sistema gráfico baseado em Canvas HTML5:

- Curva de Bézier Cúbica;
- Curva de Hermite Cúbica.

As curvas foram integradas ao sistema existente respeitando a arquitetura original e utilizando rasterização pixel a pixel.

2 Objetivos

Os principais objetivos deste trabalho foram:

- Implementar duas curvas paramétricas cúbicas;
- Integrar as curvas ao pipeline de renderização existente;
- Permitir interação através da interface gráfica;
- Manter compatibilidade com as transformações geométricas já existentes.

3 Fundamentação Matemática

3.1 Curva de Bézier

A curva de Bézier cúbica é definida por quatro pontos de controle:

$$[P_0, P_1, P_2, P_3]$$

Sua equação paramétrica é:

$$[B(t) = (1-t)^3 P_0 + 3(1-t)^2 t P_1 + 3(1-t) t^2 P_2 + t^3 P_3]$$

onde:

$$[0 \leq t \leq 1]$$

A curva passa pelos pontos P_0 e P_3 e é influenciada pelos pontos intermediários P_1 e P_2 .

3.2 Curva de Hermite

A curva de Hermite utiliza:

- Ponto inicial P_0 ;
- Ponto final P_1 ;
- Tangente inicial T_0 ;
- Tangente final T_1 .

Os polinômios base são:

$$[h_1(t) = 2t^3 - 3t^2 + 1]$$

$$[h_2(t) = -2t^3 + 3t^2]$$

$$[h_3(t) = t^3 - 2t^2 + t]$$

$$[h_4(t) = t^3 - t^2]$$

A curva é dada por:

$$[H(t) = h_1(t)P_0 + h_2(t)P_1 + h_3(t)T_0 + h_4(t)T_1]$$

4 Organização do Código

A implementação foi organizada em módulos independentes.

4.1 bezier.js

Responsável por:

- Avaliação matemática da curva;
- Rasterização;
- Visualização dos pontos de controle.

4.2 hermite.js

Responsável por:

- Avaliação matemática da curva;
- Rasterização;
- Visualização das tangentes.

4.3 render.js

Realiza a renderização dos objetos armazenados no estado da aplicação.

4.4 mouse.js

Responsável pela coleta dos cliques do usuário para criação das curvas.

5 Método de Rasterização

As curvas não são desenhadas diretamente.

Primeiramente o intervalo de parâmetro (t) é amostrado uniformemente em 100 valores.

Para cada valor de (t), calcula-se um ponto da curva.

Os pontos consecutivos são então conectados utilizando o algoritmo de Bresenham.

Dessa forma obtém-se uma aproximação discreta da curva contínua.

6 Manual de Uso

6.1 Curva de Bézier

1. Selecionar o modo Bézier.
2. Definir quatro pontos de controle.
3. A curva será criada automaticamente após o quarto clique.

6.2 Curva de Hermite

1. Selecionar o modo Hermite.
2. Definir o ponto inicial.
3. Definir o ponto final.
4. Definir a tangente inicial.
5. Definir a tangente final.

7 Resultados

As curvas foram implementadas com sucesso e integradas ao sistema gráfico.

A solução permite:

- Criação interativa das curvas;
- Seleção dos objetos;
- Aplicação de transformações geométricas;
- Visualização dos elementos de controle.

8 Conclusão

A implementação das curvas de Bézier e Hermite permitiu aplicar conceitos fundamentais de Computação Gráfica relacionados a modelagem geométrica, interpolação paramétrica e rasterização.

O sistema resultante possui caráter educacional e demonstra de forma prática a utilização de curvas paramétricas em aplicações gráficas.

9 Acesso ao Projeto

O sistema desenvolvido pode ser acessado diretamente através do navegador, sem necessidade de instalação local, por meio do endereço:

`https://tupacao.github.io/TP\_CG/`

A versão disponibilizada contém todas as funcionalidades implementadas no trabalho, incluindo:

- Desenho de retas e circunferências;
- Curvas de Bézier cúbicas;
- Curvas de Hermite cúbicas;
- Seleção de objetos;
- Transformações geométricas;
- Visualização dos elementos de controle;
- Algoritmos de clipping.

A disponibilização online facilita a validação dos resultados e a demonstração prática dos conceitos apresentados neste trabalho.